

CHAPTER 12

레이블링과 외곽선 검출

OpenCV 4로 배우는 컴퓨터 비전과 머신 러닝

12장 레이블링과 외곽선 검출

12.1 레이블링

12.2 외곽선 검출

12.1 레이블링

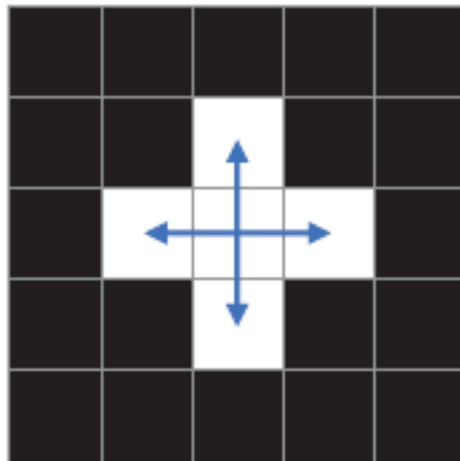
레이블링

- 이진화를 통해 배경과 객체를 구분한 후 각각의 객체를 구분하고 분석하는 작업 -> 레이블링(labeling)
- 레이블링은 영상 내에 존재하는 객체 픽셀 집합에 고유 번호를 매기는 작업으로 연결된 구성 요소 레이블링 (connected components labeling)이라고도 함
- 레이블링 기법을 이용하여 각 객체의 위치와 크기 등 정보를 추출하는 작업은 객체 인식을 위한 전처리 과정으로 자주 사용됨
- 영상의 레이블링은 일반적으로 이진화된 영상에서 수행됨
- 검은색 픽셀은 배경으로 간주하고, 흰색 픽셀은 객체로 간주함
- 하나의 객체는 한 개 이상의 인접한 픽셀로 이루어지고 하나의 객체를 구성하는 모든 픽셀에는 같은 레이블 번호가 지정됨

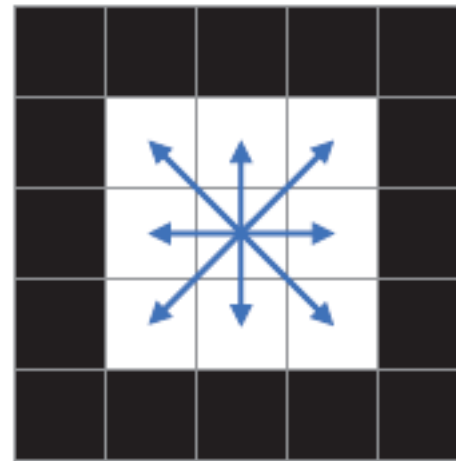
레이블링

- 특정 픽셀과 이웃한 픽셀의 연결 관계는 크게 두 가지 방식으로 정의할 수 있음
- 4-방향 연결성(4-way connectivity) : 특정 픽셀의 상하좌우로 붙어 있는 픽셀끼리 연결되어 있다고 정의하는 방법
- 8-방향 연결성(8-way connectivity) : 상하좌우로 연결된 픽셀뿐만 아니라 대각선 방향으로 인접한 픽셀도 연결되어 있다고 간주

▼ 그림 12-1 4-방향 연결성과 8-방향 연결성



(a)



(b)

레이블링

- 4-방향 연결성(4-way connectivity)을 기준으로 객체 수 : 2개
- 8-방향 연결성(8-way connectivity)을 기준으로 객체 수 : 1개

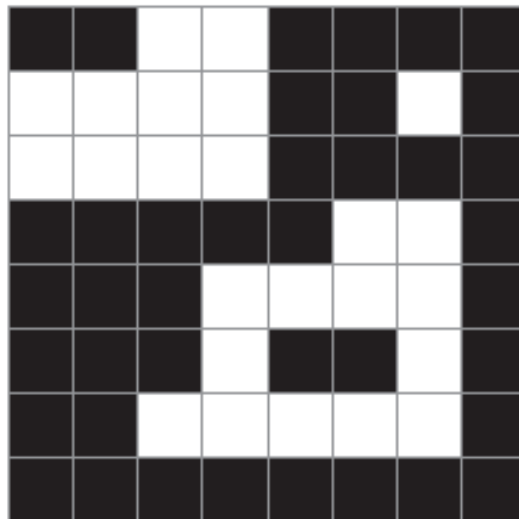
1	1		
1	1		
		2	2
		2	2

1	1		
1	1		
		1	1
		1	1

레이블링

- 이진 영상에 레이블링을 수행하면 각각의 객체 영역에 고유의 번호가 저장된 2차원 정수 행렬이 만들어짐->레이블맵(label map)이라고 부름
- 그림 12-2(a)는 8×8 크기의 이진 영상이고 그림 12-2(b)는 레이블 맵 행렬을 나타냄
- 레이블 맵 행렬에서 배경 픽셀은 0으로 설정되고 각 객체 영역에는 고유의 번호(정수)가 매겨진 것을 확인할 수 있음

▼ 그림 12-2 레이블링 알고리즘의 입력과 출력



(a)

0	0	1	1	0	0	0	0
1	1	1	1	0	0	2	0
1	1	1	1	0	0	0	0
0	0	0	0	0	3	3	0
0	0	0	3	3	3	3	0
0	0	0	3	0	0	3	0
0	0	3	3	3	3	3	0
0	0	0	0	0	0	0	0

(b)

레이블링

```
int connectedComponents(InputArray image, OutputArray labels,
                        int connectivity = 8, int ltype = CV_32S);
```

- **image** 입력 영상. CV_8UC1 또는 CV_8SC1
- **labels** 출력 레이블 맵 행렬
- **connectivity** 연결성. 8 또는 4를 지정할 수 있습니다.
- **ltype** 출력 행렬 타입. CV_32S 또는 CV_16S를 지정할 수 있습니다.
- **반환값** 레이블 개수. 반환값이 N이면 0부터 N-1까지의 레이블 번호가 존재하며, 이 중 0번 레이블은 배경을 나타냅니다. 실제 객체 개수는 N-1입니다.

- 입력은 영상이고 출력은 레이블맵을 저장한 행렬이다. 영상이 아님

코드12-1 : 레이블링 예제

```
#include <iostream>
using namespace std;
#include "opencv2/opencv.hpp"
using namespace cv;
int main()
{
    uchar data[] = {
        0, 0, 1, 1, 0, 0, 0, 0,
        1, 1, 1, 1, 0, 0, 1, 0,
        1, 1, 1, 1, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 1, 1, 0,
        0, 0, 0, 1, 1, 1, 1, 0,
        0, 0, 0, 1, 0, 0, 1, 0,
        0, 0, 1, 1, 1, 1, 1, 0,
        0, 0, 0, 0, 0, 0, 0, 0,
    };
};
```

코드12-1 : 레이블링 예제

```
Mat src = Mat(8, 8, CV_8UC1, data) * 255;

Mat labels;
int cnt = connectedComponents(src, labels);

cout << "src:\n" << src << endl;
cout << "labels:\n" << labels << endl;
cout << "number of labels: " << cnt << endl;

return 0;
}
```

코드12-1 : 레이블링 예제

▼ 그림 12-3 영상의 레이블링 예제 실행 결과

```

C:\coding\opencv\ch12\labeling\x64\Debug\labeling.exe
src:
[ 0, 0, 255, 255, 0, 0, 0, 0;
 255, 255, 255, 255, 0, 0, 255, 0;
 255, 255, 255, 255, 0, 0, 0, 0;
 0, 0, 0, 0, 0, 255, 255, 0;
 0, 0, 0, 255, 255, 255, 255, 0;
 0, 0, 0, 255, 0, 0, 255, 0;
 0, 0, 255, 255, 255, 255, 255, 0;
 0, 0, 0, 0, 0, 0, 0, 0]
labels:
0, 0, 1, 1, 0, 0, 0, 0;
1, 1, 1, 1, 0, 0, 2, 0;
1, 1, 1, 1, 0, 0, 0, 0;
0, 0, 0, 0, 0, 3, 3, 0;
0, 0, 0, 3, 3, 3, 3, 0;
0, 0, 0, 3, 0, 0, 3, 0;
0, 0, 3, 3, 3, 3, 3, 0;
0, 0, 0, 0, 0, 0, 0, 0]
number of labels: 4

```

레이블링 응용

- 기본적인 레이블링 동작은 입력 영상으로부터 레이블 맵을 생성하는 것임
- 보통 레이블링을 수행한 후에는 각각의 객체 영역이 어느 위치에 어느 정도의 크기로 존재하는지 확인할 필요가 있음
- 이러한 작업을 사용자가 for 반복문 등을 이용하여 직접 구현하기는 꽤 번거로움
- 다행히 OpenCV는 레이블 맵과 각 객체 영역의 통계 정보를 한꺼번에 반환하는 `connectedComponentsWithStats()` 함수를 제공함

레이블링 응용

```
int connectedComponentsWithStats(InputArray image, OutputArray labels,
                                OutputArray stats, OutputArray centroids,
                                int connectivity = 8, int ltype = CV_32S);
```

- **image** 입력 영상. CV_8UC1 또는 CV_8SC1
- **labels** 출력 레이블 맵 행렬
- **stats** 각각의 레이블 영역에 대한 통계 정보를 담은 행렬. CV_32S
- **centroids** 각각의 레이블 영역의 무게 중심 좌표 정보를 담은 행렬. CV_64F
- **connectivity** 연결성. 8 또는 4를 지정할 수 있습니다.
- **ltype** 출력 행렬 타입. CV_32S 또는 CV_16S를 지정할 수 있습니다.
- **반환값** 레이블 개수. 반환값이 N이면 0부터 N-1까지의 레이블 번호가 존재하며, 이 중 0번 레이블은 배경을 나타냅니다. 실제 객체 개수는 N-1입니다.

- stats, centroids 행렬의 원소의 자료형을 잘 기억할 것

레이블링 응용

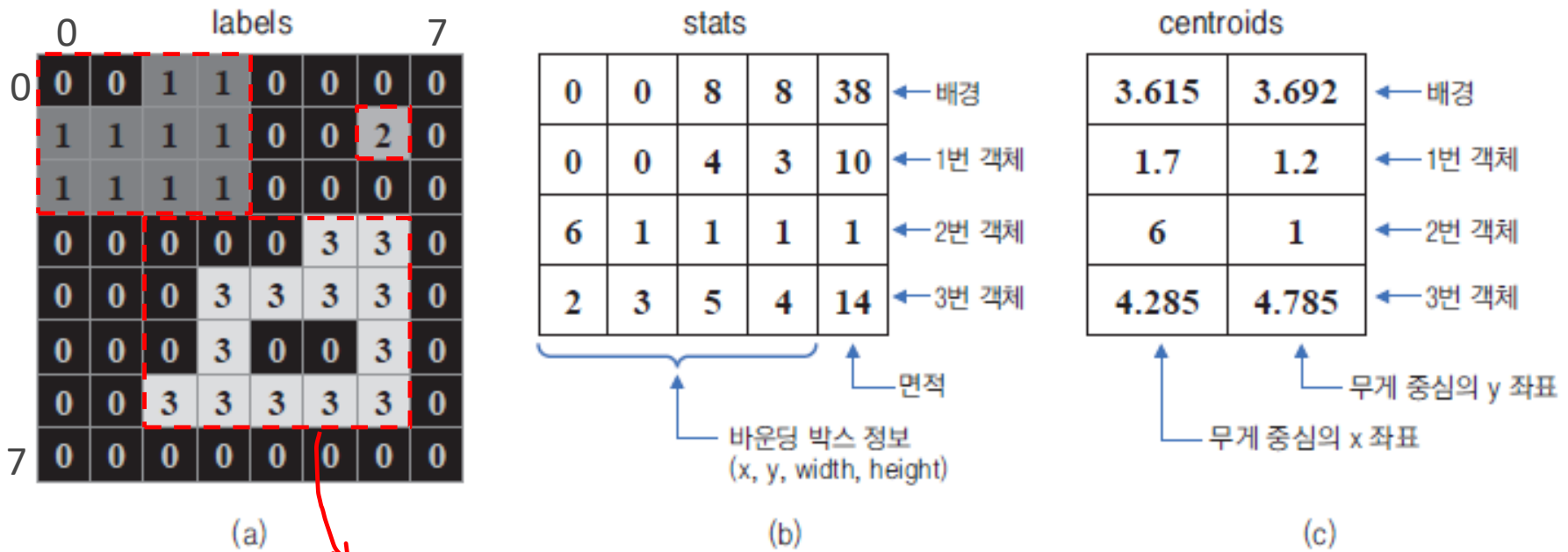
- `connectedComponentsWithStats()` 함수의 인자 구성은 `connectedComponents()` 함수 인자에 `stats`와 `centroids`가 추가됨
- 보통 `stats`와 `centroids` 인자에는 `Mat` 자료형 변수를 지정함
- 입력 영상 `src`가 있을 때 `connectedComponentsWithStats()` 함수를 이용하여 레이블링을 수행하려면 다음과 같이 코드를 작성함

```
Mat labels, stats, centroids;  
connectedComponentsWithStats(src, labels, stats, centroids);
```

레이블링 응용

- 그림 12-2에서 사용한 8×8 영상에 대해 앞의 예제 코드를 수행했을 때 생성되는 labels, stats, centroids 행렬을 그림 12-4에 나타냄

▼ 그림 12-4 connectedComponentsWithStats() 함수의 출력 행렬 분석



바운딩박스 : 객체를 포함하는 가장 작은 직사각형

레이블링 응용

- stats 행렬에서 두 번째 행 원소 값이 [0, 0, 4, 3, 10]으로 저장됨 -> 1번 객체를 감싸는 바운딩 박스가 (0, 0) 좌표에서 시작하여 가로 크기가 4, 세로 크기가 3인 사각형이고 객체 영역의 픽셀 개수(면적)가 10임을 나타냄
- centroids 행렬에서 두 번째 행 원소 값이 [1.7, 1.2]로 저장된 것은 1번 영역의 무게 중심 좌표가 (1.7, 1.2)라는 것을 의미함
- 무게중심 x좌표 = (객체영역에 포함된 모든픽셀의 x좌표의 합) / 객체영역의 총픽셀수
- 무게중심 y좌표 = (객체영역에 포함된 모든픽셀의 y좌표의 합) / 객체영역의 총픽셀수

$$(1.7, 1.2) = \left(\frac{2+3+0+1+2+3+0+1+2+3}{10}, \frac{0+0+1+1+1+1+2+2+2+2}{10} \right)$$

레이블링 응용

- stats, centroids 의 원소를 참조하는 방법 -> Mat::at() 함수를 이용하여 참조
- stats행렬의 원소의 자료형은 **CV_32S**이고 centroids 행렬의 원소의 자료형은 **CV_64F**

stats				
0	0	8	8	38
0	0	4	3	10
6	1	1	1	1
2	3	5	4	14

← 배경
← 1번 객체
← 2번 객체
← 3번 객체

centroids	
3.615	3.692
1.7	1.2
6	1
4.285	4.785

← 배경
← 1번 객체
← 2번 객체
← 3번 객체

`int x = stats.at<int>(행, 열)` `double x = centroids.at<double>(행, 열)`

예제: 통계정보 참조하기

```
#include <iostream>
using namespace std;
#include "opencv2/opencv.hpp"
using namespace cv;
int main()
{
    Mat src = imread("labelex.png", IMREAD_GRAYSCALE);
    if (src.empty()) { cerr << "Image load failed!" << endl; return -1;}
    Mat bin;
    threshold(src, bin, 0, 255, THRESH_BINARY_INV | THRESH_OTSU);

    Mat labels, stats, centroids;
    int cnt = connectedComponentsWithStats(bin, labels, stats, centroids);

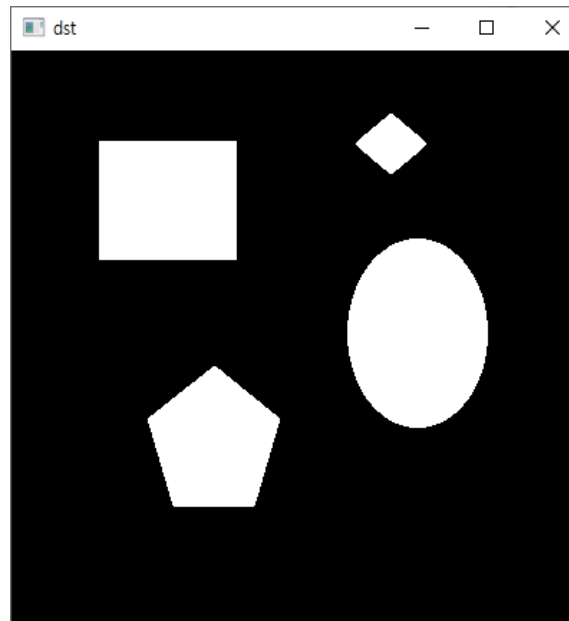
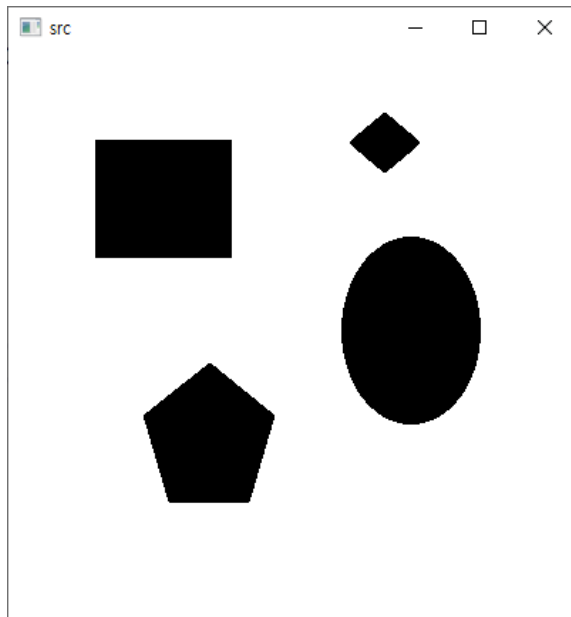
    for (int j = 0; j < stats.rows; j++)
        cout << j << "번객체:" << "(x,y):" << setw(5) << stats.at<int>(j, 0)
            << setw(5) << stats.at<int>(j, 1) << endl;
    imshow("src", src);
    imshow("dst", bin);
    return 0;
}
```

예제: 통계정보 참조하기

```

D:\Users\2sungryul\Dropbox\Lecture\2...
0번 객체 : (x,y) :    0    0
1번 객체 : (x,y) :  240   44
2번 객체 : (x,y) :   61   63
3번 객체 : (x,y) :  234  131
4번 객체 : (x,y) :   95  220

```



코드12-2 : 바운딩 박스 그리기

- `connectedComponentsWithStats()` 함수를 사용하여 실제 영상에 레이블링을 수행하고 검출된 객체의 위치와 크기를 화면에 표시하는 예제
- 입력 영상을 이진화한 후 레이블링을 수행함
- 레이블링에 의해 얻은 객체 통계 정보를 이용하여 각 객체를 감싸는 바운딩 박스를 노란색 사각형으로 표시함

코드12-2 : 바운딩 박스 그리기

```
#include <iostream>
using namespace std;
#include "opencv2/opencv.hpp"
using namespace cv;

int main()
{
    Mat src = imread("keyboard.bmp", IMREAD_GRAYSCALE);
    if (src.empty()) { cerr << "Image load failed!" << endl; return -1;}

    Mat bin;
    threshold(src, bin, 0, 255, THRESH_BINARY | THRESH_OTSU);

    Mat labels, stats, centroids;
    int cnt = connectedComponentsWithStats(bin, labels, stats, centroids);

    Mat dst;
    cvtColor(src, dst, COLOR_GRAY2BGR);
```

코드12-2 : 바운딩 박스 그리기

```
for (int i = 1; i < cnt; i++) {
    int* p = stats.ptr<int>(i);    //stats객체의 i행의 시작주소를 리턴
    if (p[4] < 20) continue;
    rectangle(dst, Rect(p[0], p[1], p[2], p[3]), Scalar(0, 255, 255));
}
imshow("src", src);
imshow("dst", dst);
waitKey();
destroyAllWindows();
return 0;
}
```

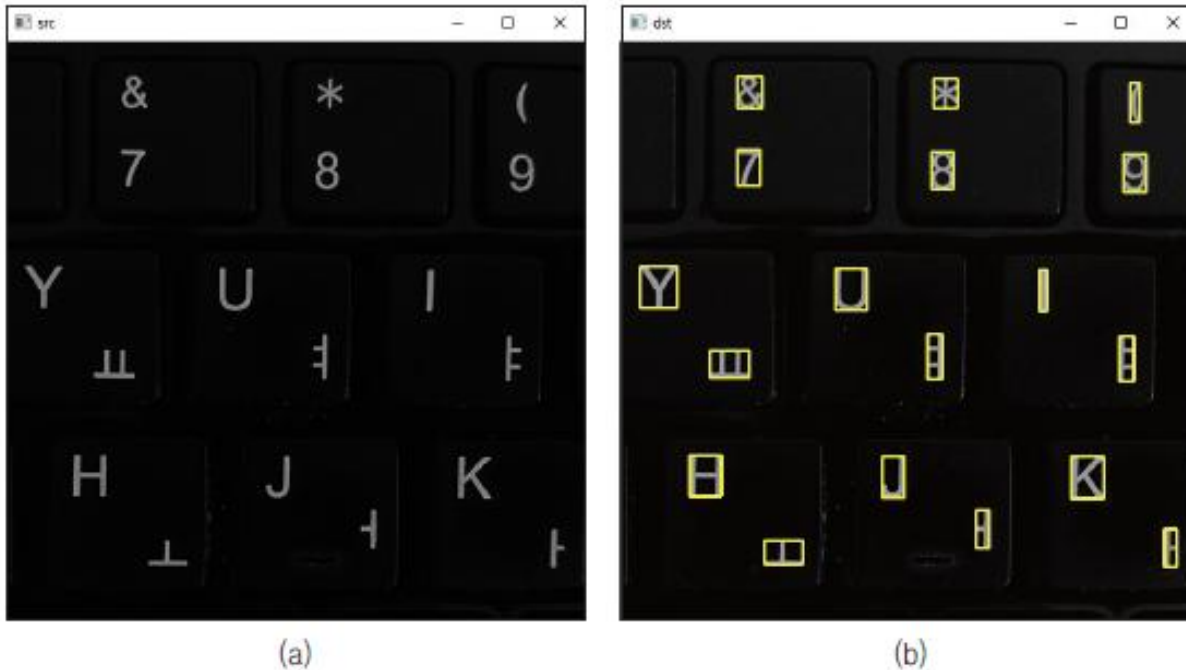
	P[0]	P[1]	P[2]	P[3]	P[4]	
stats	0	0	4	3	10	← 1번 객체
	6	1	1	1	1	← 2번 객체
	2	3	5	4	14	← 3번 객체

면적

바운딩 박스 정보
(x, y, width, height)

코드12-2 : 바운딩 박스 그리기

▼ 그림 12-5 레이블링을 이용하여 객체의 바운딩 박스 그리기 실행 결과



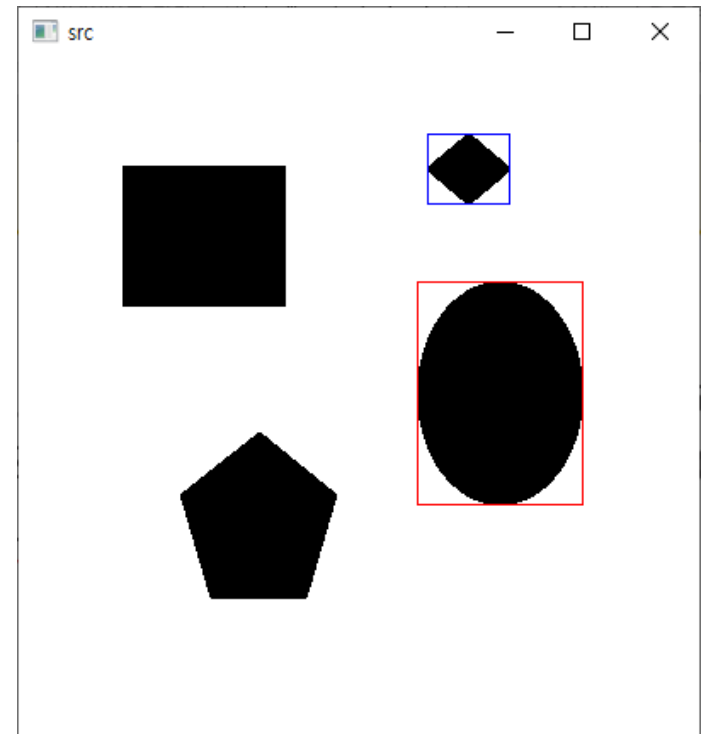
- 그림 12-5에서 src는 입력으로 사용한 keyboard.bmp 영상임
- dst는 키보드에서 흰색 글자만을 찾아서 노란색 사각형으로 표시한 결과 영상임, 키보드에 적힌 각 문자를 제대로 구분하여 표시한 것을 확인 할 수 있음

실습과제1

- labelex.png 영상파일(컬러영상)에서 레이블링을 수행하고 객체의 면적이 최대, 최소인 객체를 찾고 아래처럼 콘솔창에 정보를 출력하고 원본영상에 바운딩 박스를 그려주세요.
- 힌트 : 컬러->그레이->이진화(반전)->레이블링

```

D:\Users\2sungryul\Dropbox\Lecture\2022년1학기\컴...
면적이 최대인 객체의 레이블:3
무게중심:(x,y): 282.532 196.529
면적이 최소인 객체의 레이블:1
무게중심:(x,y): 264.02 64.5448
  
```

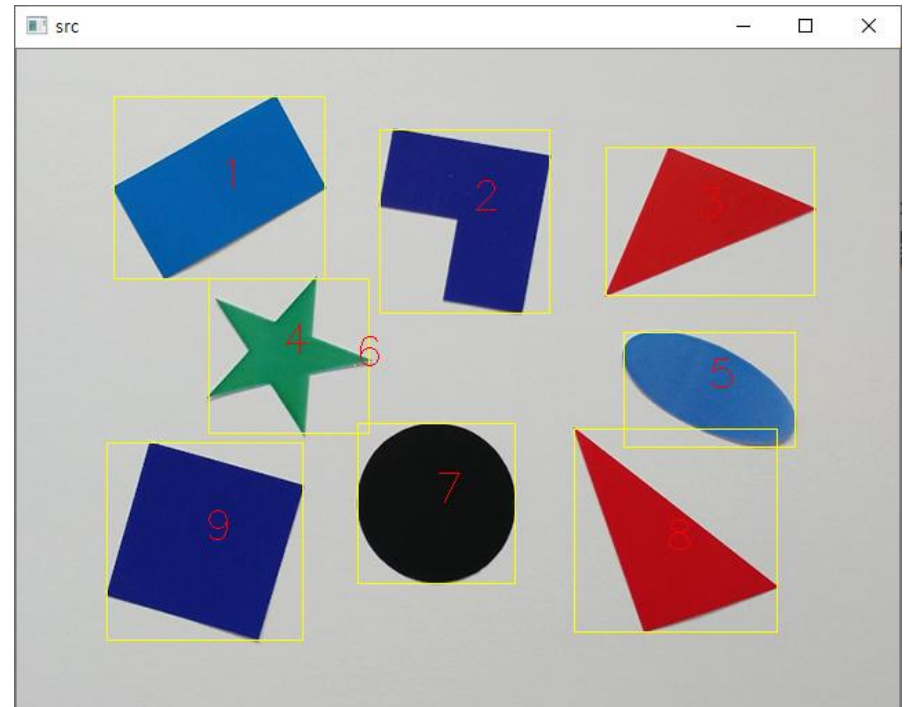


실습과제2

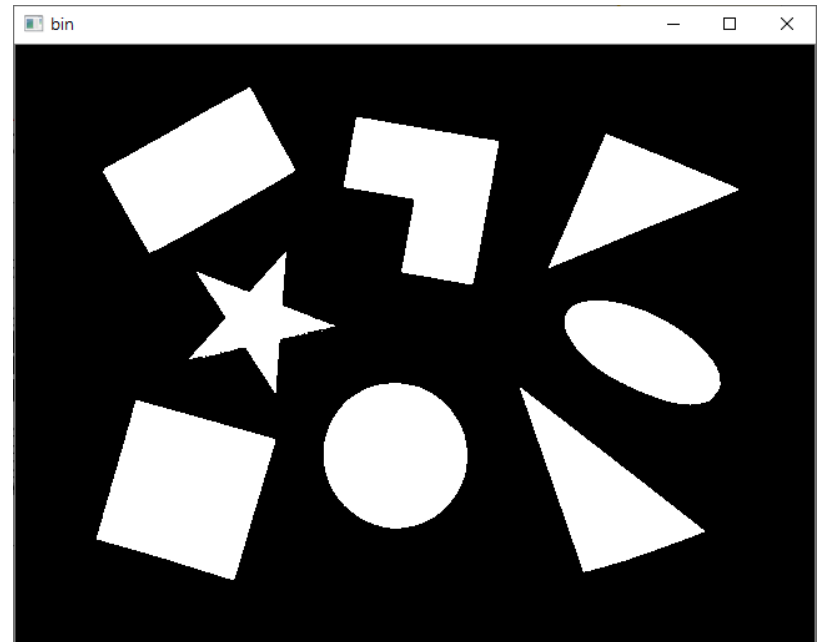
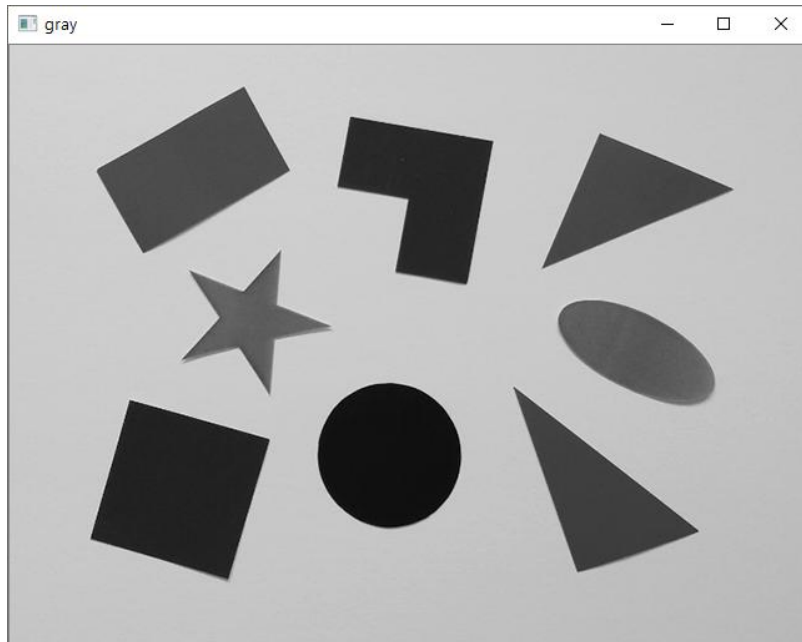
- polygon.bmp 영상파일에서 레이블링을 수행하고 다음 처럼 콘솔창에 아래와 같은 정보를 출력하고 영상원본에 바운딩 박스와 레이블값을 그려주는 코드를 작성하시오. color는 무게중심 좌표에서 원본 영상의 컬러를 출력할 것

```

D:\Users\W2sungryul\Dropbox\Lecture\2021년1학기\... - □ ×
갯수10
label x y width height area color[B,G,R]
1 70 34 154 133 10499 [164, 90, 3]
2 263 58 124 134 10157 [116, 30, 19]
3 427 71 152 108 6906 [23, 22, 183]
4 139 166 117 113 4381 [90, 132, 21]
5 440 205 125 84 6550 [165, 96, 38]
6 245 229 1 1 1 [130, 132, 117]
7 247 271 115 117 10523 [17, 18, 16]
8 404 275 148 148 8282 [23, 12, 178]
9 65 285 143 144 13563 [112, 25, 17]
  
```



실습과제2



실습과제3

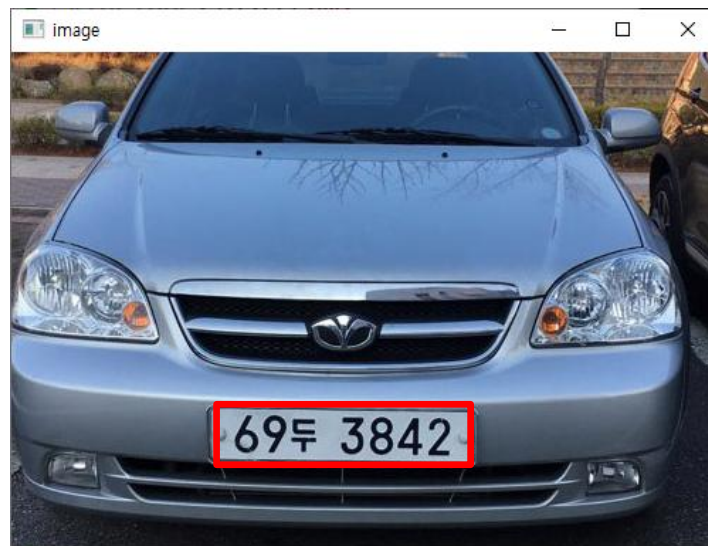
- polygon.bmp 영상파일에서 레이블링을 수행하고 각 객체의 무게중심을 직접 계산하여 아래 처럼 화면에 출력하시오.
- 레이블링 함수의 결과와 같은지 비교하시오.
- 무게중심 x좌표 = (객체영역에 포함된 모든점의 x좌표의 합) / 객체영역의 총픽셀수
- 무게중심 y좌표 = (객체영역에 포함된 모든점의 y좌표의 합) / 객체영역의 총픽셀수

```

Microsoft Visual Studio 디버그 콘솔
1번 객체의 무게중심: (146.742, 100.313)
2번 객체의 무게중심: (330.783, 116.606)
3번 객체의 무게중심: (493.084, 121.446)
4번 객체의 무게중심: (192.264, 220.614)
5번 객체의 무게중심: (501.927, 245.667)
6번 객체의 무게중심: (245, 229)
7번 객체의 무게중심: (303.828, 328.685)
8번 객체의 무게중심: (470.336, 362.324)
9번 객체의 무게중심: (136.025, 355.962)
  
```

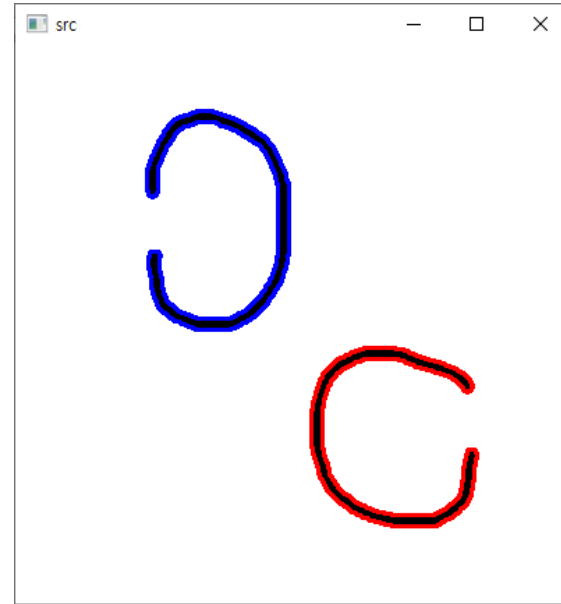
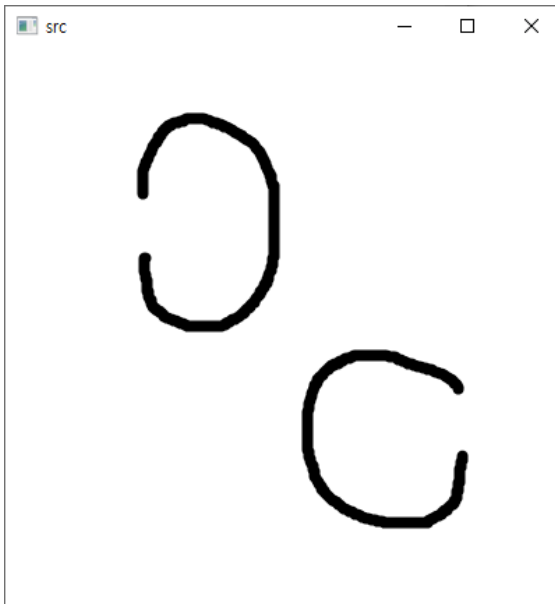
실습과제4

- 11장-2의 실습과제4의 결과를 이용하여 자동차의 번호판 영역을 검출하여 다음처럼 빨간색으로 표시하라.



실습과제5

- char_c.png 영상에서 열려있는 방향을 구하시오. 열린방향이 왼쪽이면 외곽선을 파란색으로 그려주고 열린방향이 오른쪽이면 외곽선을 빨간색으로 그리시오.



D:\Users\2sungryul\Dropbox\Lecture\2022년1학기\컴...
0번째 문자(빨간색):오른쪽으로 열림
1번째 문자(파란색):왼쪽으로 열림

과제 제출 방법

- 소스파일, 라인단위 코드설명, 실행결과를 하나의 PDF 파일로 작성하여 과제게시판에 제출하시오. 이외 양식은 0점 처리함
- 제출기한은 과제 게시판을 참고할 것.